

Question 1

Implementing Multiple Linear Regression (from scratch)

```
In [34]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
```

Loading Dataset

```
In [35]: df = pd.read_csv('house_sales.csv')

print("Shape of dataset:", df.shape)
print("\nFirst few rows:")
df.head()
```

Shape of dataset: (21613, 21)

First few rows:

```
Out[35]:
```

	id	date	price	bedrooms	bathrooms	sqft_liv
0	7129300520	20141013T000000	221900.0	3	1.00	1
1	6414100192	20141209T000000	538000.0	3	2.25	2
2	5631500400	20150225T000000	180000.0	2	1.00	
3	2487200875	20141209T000000	604000.0	4	3.00	1
4	1954400510	20150218T000000	510000.0	3	2.00	1

5 rows × 21 columns

```
In [36]: print("Dataset Info:")
df.info()
print("\nBasic Statistics:")
df.describe()
```

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

RangeIndex: 21613 entries, 0 to 21612

Data columns (total 21 columns):

#	Column	Non-Null Count	Dtype
0	id	21613 non-null	int64
1	date	21613 non-null	object
2	price	21613 non-null	float64
3	bedrooms	21613 non-null	int64
4	bathrooms	21613 non-null	float64
5	sqft_living	21613 non-null	int64
6	sqft_lot	21613 non-null	int64
7	floors	21613 non-null	float64
8	waterfront	21613 non-null	int64
9	view	21613 non-null	int64
10	condition	21613 non-null	int64
11	grade	21613 non-null	int64
12	sqft_above	21613 non-null	int64
13	sqft_basement	21613 non-null	int64
14	yr_built	21613 non-null	int64
15	yr_renovated	21613 non-null	int64
16	zipcode	21613 non-null	int64
17	lat	21613 non-null	float64
18	long	21613 non-null	float64
19	sqft_living15	21613 non-null	int64
20	sqft_lot15	21613 non-null	int64

dtypes: float64(5), int64(15), object(1)

memory usage: 3.5+ MB

Basic Statistics:

```
Out[36]:
```

	id	price	bedrooms	bathrooms	sqft_liv
count	2.161300e+04	2.161300e+04	21613.000000	21613.000000	21613.000
mean	4.580302e+09	5.400881e+05	3.370842	2.114757	2079.899
std	2.876566e+09	3.671272e+05	0.930062	0.770163	918.440
min	1.000102e+06	7.500000e+04	0.000000	0.000000	290.000
25%	2.123049e+09	3.219500e+05	3.000000	1.750000	1427.000
50%	3.904930e+09	4.500000e+05	3.000000	2.250000	1910.000
75%	7.308900e+09	6.450000e+05	4.000000	2.500000	2550.000
max	9.900000e+09	7.700000e+06	33.000000	8.000000	13540.000

Feature Selection and Target Variable

```
In [37]: feature_cols = ['bedrooms', 'bathrooms', 'sqft_living', 'floors', 'lat', 'long']
         target_col    = 'price'

# dropping nulls if there
data = df[feature_cols + [target_col]].dropna()
```

```
print("Rows after cleaning:", len(data))
print("\nMissing values:")
print(data.isnull().sum())
```

Rows after cleaning: 21613

Missing values:

```
bedrooms      0
bathrooms     0
sqft_living   0
floors        0
waterfront    0
price         0
dtype: int64
```

```
In [38]: # Counting 0's and 1's
print("Waterfront value counts:")
print(data['waterfront'].value_counts())

print("\nCorrelation with price:")
print(data.corr()['price'].sort_values(ascending=False))
```

Waterfront value counts:

```
waterfront
0      21450
1        163
Name: count, dtype: int64
```

Correlation with price:

```
price          1.000000
sqft_living    0.702035
bathrooms      0.525138
bedrooms       0.308350
waterfront     0.266369
floors         0.256794
Name: price, dtype: float64
```

Data Preprocessing

```
In [39]: X_raw = data[feature_cols].values.astype(float)
y        = data[target_col].values.astype(float)

print("X shape:", X_raw.shape)
print("y shape:", y.shape)
```

```
X shape: (21613, 5)
y shape: (21613,)
```

```
In [40]: # Standard Scaling using library

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_norm = scaler.fit_transform(X_raw)

print("Feature means:", scaler.mean_)
```

```
print("Feature stds :", scaler.scale_)
print("\nStandardized X (first 3 rows):")
print(X_norm[:3])
```

Feature means: [3.37084162e+00 2.11475732e+00 2.07989974e+03 1.49430898e+00

7.54175728e-03]

Feature stds : [9.30040315e-01 7.70145340e-01 9.18419649e+02 5.39976403e-01

8.65151962e-02]

Standardized X (first 3 rows):

```
[[-0.39873715 -1.44746357 -0.97983502 -0.915427 -0.08717263]
 [-0.39873715  0.1756067  0.53363434  0.93650577 -0.08717263]
 [-1.47395936 -1.44746357 -1.42625404 -0.915427 -0.08717263]]
```

Train-test split using sklearn

```
In [41]: X_train, X_test, y_train, y_test = train_test_split(
        X_norm, y, test_size=0.2, random_state=42
    )

print(f"Training samples : {len(X_train)}")
print(f"Testing samples : {len(X_test)}")
```

Training samples : 17290

Testing samples : 4323

```
In [42]: ones_train = np.ones((X_train.shape[0], 1))
        ones_test = np.ones((X_test.shape[0], 1))

        X_train_b = np.hstack([ones_train, X_train])
        X_test_b = np.hstack([ones_test, X_test])

        print("X_train_b shape:", X_train_b.shape)
        print("X_test_b shape :", X_test_b.shape)
```

X_train_b shape: (17290, 6)

X_test_b shape : (4323, 6)

Implementing Multiple Linear Regression

$$\theta = (X^T X)^{-1} X^T y$$

```
In [43]: def normal_equation(X, y):

        XtX = X.T @ X
        Xty = X.T @ y
        theta = np.linalg.pinv(XtX) @ Xty
        return theta

        theta = normal_equation(X_train_b, y_train)

        labels = ['bias'] + feature_cols
        print("Learned Weights:")
```

```
for name, val in zip(labels, theta):
    print(f" {name:15s}: {val:.4f}")
```

Learned Weights:

```
bias      : 539690.4887
bedrooms  : -46051.5156
bathrooms : 8288.9894
sqft_living : 270708.3219
floors    : 907.9322
waterfront : 66516.7336
```

MODEL EVALUATION

```
In [45]: from sklearn.metrics import mean_squared_error, mean_absolute_error

def model_mlr_036(X, theta):
    return X @ theta

pred_mlr_036 = model_mlr_036(X_test_b, theta)

mse_lib = mean_squared_error(y_test, pred_mlr_036)
mae_lib = mean_absolute_error(y_test, pred_mlr_036)
r2_lib = r2_score(y_test, pred_mlr_036)

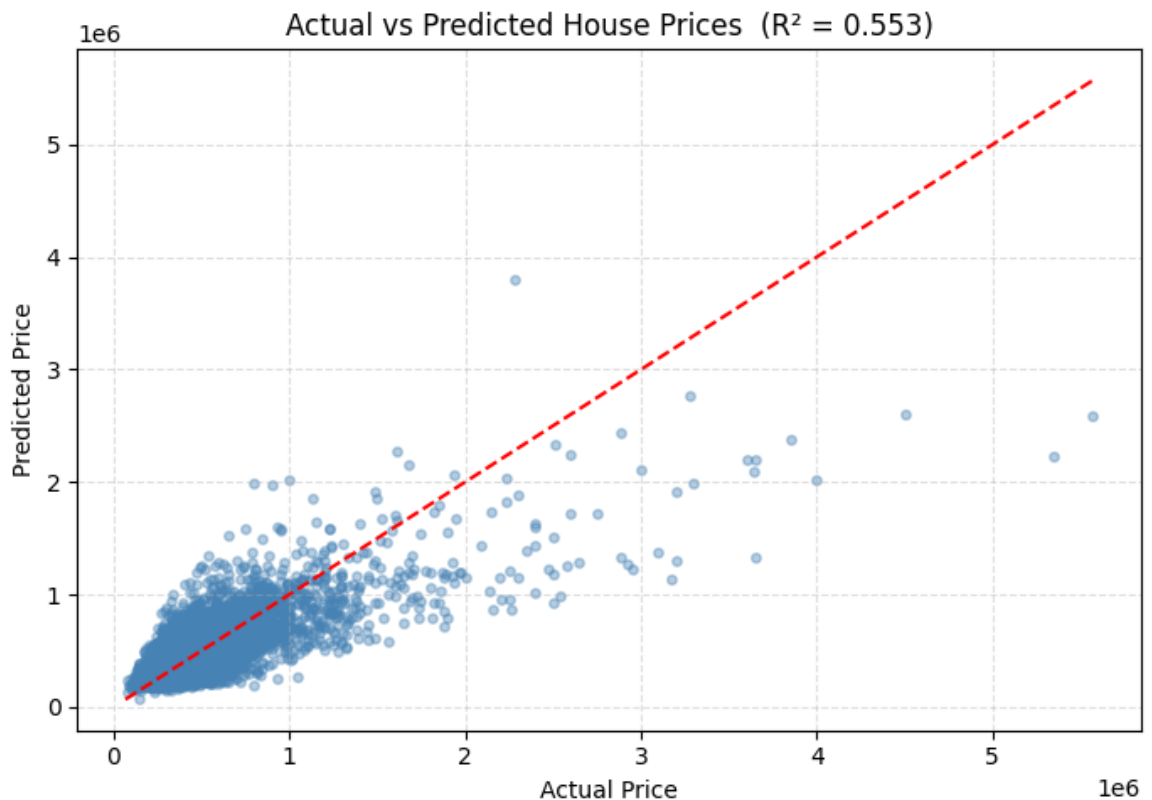
print("Model Evaluation")
print(f" Test MSE : {mse_lib:.2f}")
print(f" Test RMSE : {mse_lib**0.5:.2f}")
print(f" Test MAE : {mae_lib:.2f}")
print(f" R2 Score : {r2_lib:.4f}")
```

Model Evaluation

```
Test MSE : 67594154591.99
Test RMSE : 259988.76
Test MAE : 170351.05
R2 Score : 0.5529
```

Vizualization

```
In [46]: # Actual vs Predicted scatter plot
plt.figure(figsize=(7, 5))
plt.scatter(y_test, pred_mlr_036, alpha=0.4, color='steelblue', s=100)
min_val = min(y_test.min(), pred_mlr_036.min())
max_val = max(y_test.max(), pred_mlr_036.max())
plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=1)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Price')
plt.title(f'Actual vs Predicted House Prices (R2 = {r2_lib:.3f})')
plt.grid(True, linestyle='--', alpha=0.4)
plt.tight_layout()
plt.show()
```



Model Prediction

- Bedrooms: 3
- Bathrooms: 2
- Sqft Living: 2000
- Floors: 2
- Waterfront: 0

```
In [47]: # Predicted items inside an array
new_house = np.array([3, 2, 2000, 2, 0], dtype=float)

new_house_norm = scaler.transform(new_house.reshape(1, -1)).flatten

new_house_b = np.hstack([[1.0], new_house_norm])

print("Standardized input:", new_house_norm)
print("Input with bias   :", new_house_b)

Standardized input: [-0.39873715 -0.14900736 -0.08699698  0.93650577
-0.08717263]
Input with bias   : [ 1.          -0.39873715 -0.14900736 -0.08699698
 0.93650577 -0.08717263]
```

```
In [48]: pred_price = model_mlr_036(new_house_b, theta)

print("Predicted House Price")
print(f" Bedrooms    : 3")
print(f" Bathrooms    : 2")
```

```
print(f"  Sqft Living : 2000")
print(f"  Floors      : 2")
print(f"  Waterfront   : 0 (No)")
print(f"  Predicted Price : $ {pred_price:,.2f}")
```

Predicted House Price

```
Bedrooms      : 3
Bathrooms     : 2
Sqft Living   : 2000
Floors        : 2
Waterfront    : 0 (No)
Predicted Price : $ 528,318.86
```

Question 2

Logistic Regression to predict whether a tumor is malignant or benign

```
In [14]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, confusion_matrix,
    classification_report, ConfusionMatrixDisplay
)
```

Loading Dataset

```
In [15]: bc = pd.read_csv('breast_cancer.csv')

print("Shape:", bc.shape)
print("\nDiagnosis counts:")
print(bc['diagnosis'].value_counts())

bc.head()
```

Shape: (569, 33)

Diagnosis counts:

diagnosis

B 357

M 212

Name: count, dtype: int64

Out[15]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_
0	842302	M	17.99	10.38	122.80	
1	842517	M	20.57	17.77	132.90	
2	84300903	M	19.69	21.25	130.00	
3	84348301	M	11.42	20.38	77.58	
4	84358402	M	20.29	14.34	135.10	

5 rows x 33 columns

Data Preprocessing

```
In [26]: from sklearn.preprocessing import LabelEncoder

feature_cols = ['radius_mean', 'texture_mean', 'perimeter_mean',
                'area_mean', 'smoothness_mean']

X = bc[feature_cols].values

# Label Encoding - B=0, M=1
le = LabelEncoder()
y = le.fit_transform(bc['diagnosis'])
```

Train-Test Split

```
In [17]: X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.20,
        random_state=42,
        stratify=y,          # keeps class ratio balanced in both splits
    )

print(f"Train: {len(X_train)} samples")
print(f"Test : {len(X_test)} samples")
```

Train: 455 samples

Test : 114 samples

Feature Scaling

```
In [27]: scaler_bc = StandardScaler()
X_train_s = scaler_bc.fit_transform(X_train) # fit on train only
X_test_s   = scaler_bc.transform(X_test)     # apply same scale to
print("Scaling Completed")
```

Scaling Completed

Training Logistic Regression Model


```
In [28]: model_LR_036 = LogisticRegression(C=1.0, max_iter=10000, solver='lbfgs')
model_LR_036.fit(X_train_s, y_train)

print("Model trained successfully.")
```

Model trained successfully.

Model Evaluation

```
In [30]: y_pred_bc = model_LR_036.predict(X_test_s)
y_prob_bc = model_LR_036.predict_proba(X_test_s)[:, 1]

acc = accuracy_score(y_test, y_pred_bc)
print(f"Test Accuracy : {acc * 100:.2f}%")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_bc, target_names=['Benign', 'Malignant']))
```

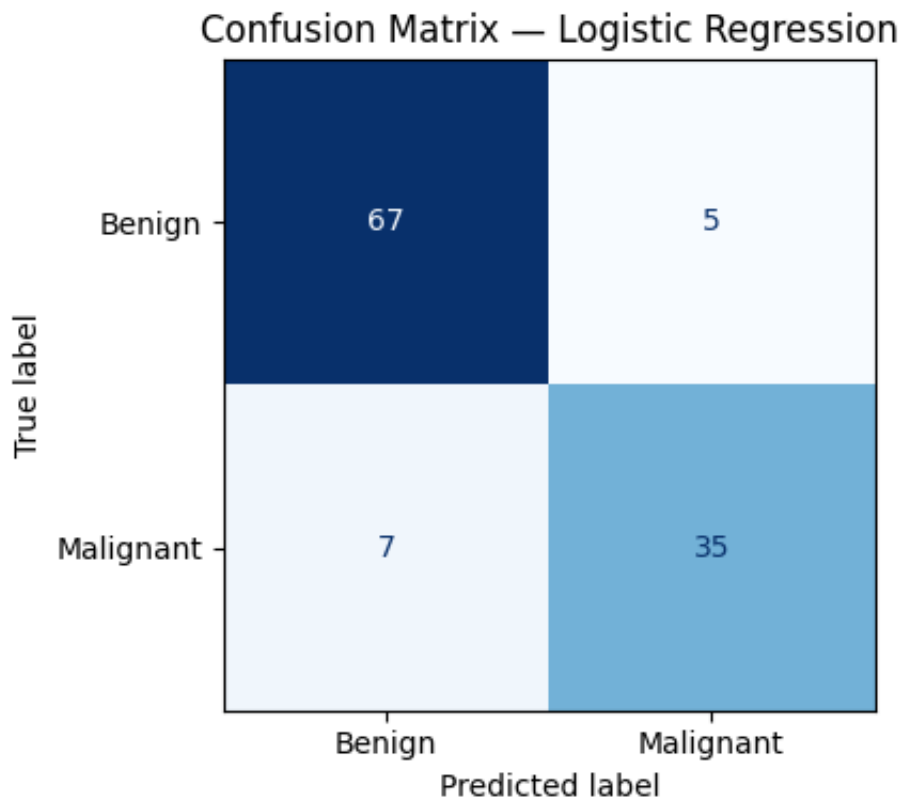
Test Accuracy : 89.47%

Classification Report:

	precision	recall	f1-score	support
Benign	0.91	0.93	0.92	72
Malignant	0.88	0.83	0.85	42
accuracy			0.89	114
macro avg	0.89	0.88	0.89	114
weighted avg	0.89	0.89	0.89	114

```
In [21]: cm = confusion_matrix(y_test, y_pred_bc)

fig, ax = plt.subplots(figsize=(5, 4))
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['Benign', 'Malignant'])
disp.plot(ax=ax, colorbar=False, cmap='Blues')
ax.set_title('Confusion Matrix - Logistic Regression')
plt.tight_layout()
plt.show()
```



Prediction

Given:

- Radius mean: 14.5
- Texture mean: 20.0
- Perimeter mean: 90.0
- Area mean: 600.0
- Smoothness mean: 0.1

```
In [32]: # New patient data
new_tumor = np.array([[14.5, 20.0, 90.0, 600.0, 0.1]])

# Scale using the same scaler fitted on training data
new_tumor_s = scaler_bc.transform(new_tumor)

# Predict
pred_label = model_LR_036.predict(new_tumor_s)[0]
pred_prob = model_LR_036.predict_proba(new_tumor_s)[0]
label_map = {0: 'Benign', 1: 'Malignant'}

print("Tumor Diagnosis Prediction")

print(f"Radius Mean      : 14.5")
print(f"Texture Mean      : 20.0")
print(f"Perimeter Mean     : 90.0")
print(f"Area Mean          : 600.0")
print(f"Smoothness Mean    : 0.1")
```

```
print(f"Prediction      : {label_map[pred_label]}")
print(f"Prob (Benign)   : {pred_prob[0] * 100:.2f}%")
print(f"Prob (Malignant): {pred_prob[1] * 100:.2f}%")
```

Tumor Diagnosis Prediction

Radius Mean : 14.5

Texture Mean : 20.0

Perimeter Mean : 90.0

Area Mean : 600.0

Smoothness Mean : 0.1

Prediction : Benign

Prob (Benign) : 57.88%

Prob (Malignant): 42.12%